

Operating Systems Lab

Lab 1: Processes and Threads in Python

Subject:	Operating Systems
Lab No:	Lab 1
Topic:	Process and Thread Creation & Termination
Language:	Python 3
Platform:	Linux (Ubuntu)

Question 1: Create and Terminate a Process

Objective

To create a child process using Python's multiprocessing module and observe its execution and termination.

Algorithm

1. Start the program.
2. Import the **Process** class from the **multiprocessing** module.
3. Import the **time** module.
4. Define a function **worker()** to act as the child process body.
5. Inside **worker()**, print a message: 'Child process started'.
6. Pause execution for 2 seconds using **time.sleep(2)**.
7. Print a message: 'Child process terminated'.
8. In the main block (if **__name__ == '__main__'**), print 'Main process started'.
9. Create a process object: **p = Process(target=worker)**.
10. Start the process using **p.start()**.
11. Wait for the process to finish using **p.join()**.
12. Print 'Main process terminated'.
13. End the program.

Source Code — process_lab.py

```
from multiprocessing import Process
import time

def worker():
    print("Child process started")
    time.sleep(2)
```

```

        print("Child process terminated")

if __name__ == "__main__":
    print("Main process started")

    p = Process(target=worker)    # Create process
    p.start()                    # Start process

    p.join()                     # Wait until process finishes

    print("Main process terminated")

```

Code Explanation

Line / Statement	Explanation
from multiprocessing import Process	Imports the Process class to create new processes.
import time	Imports the time module to use sleep().
def worker():	Defines the function that runs inside the child process.
time.sleep(2)	Simulates work by pausing execution for 2 seconds.
if __name__ == '__main__':	Ensures this block only runs when script is executed directly.
p = Process(target=worker)	Creates a new child process with worker() as its function.
p.start()	Starts the child process — it begins running worker().
p.join()	Main process waits here until the child process finishes.

Expected Output

```

Main process started
Child process started
Child process terminated
Main process terminated

```

Conclusion

The program successfully created a child process using Python's multiprocessing module. The child process executed the worker() function independently and terminated after completing its task. The main process waited for the child to finish using join() before printing its final message.

Question 2: Create and Terminate a Thread

Objective

To create a thread using Python's threading module and observe its execution and termination.

Algorithm

1. Start the program.
2. Import the **Thread** class from the **threading** module.
3. Import the **time** module.
4. Define a function **worker()** to act as the thread's task.
5. Inside **worker()**, print a message: 'Thread started'.
6. Pause execution for 2 seconds using **time.sleep(2)**.
7. Print a message: 'Thread terminated'.
8. In the main program, print 'Main program started'.
9. Create a thread object: **t = Thread(target=worker)**.
10. Start the thread using **t.start()**.
11. Wait for the thread to finish using **t.join()**.
12. Print 'Main program terminated'.
13. End the program.

Source Code — thread_lab.py

```
from threading import Thread
import time

def worker():
    print("Thread started")
    time.sleep(2)
    print("Thread terminated")

print("Main program started")

t = Thread(target=worker)    # Create thread
t.start()                   # Start thread

t.join()                     # Wait for thread to finish

print("Main program terminated")
```

Code Explanation

Line / Statement	Explanation
from threading import Thread	Imports the Thread class to create new threads.
import time	Imports the time module to use sleep().
def worker():	Defines the function that the thread will execute.
time.sleep(2)	Simulates work by pausing the thread for 2 seconds.
t = Thread(target=worker)	Creates a new thread with worker() as its target function.
t.start()	Starts the thread — worker() begins executing in parallel.

```
t.join()
```

Main program waits here until the thread finishes.

Expected Output

```
Main program started
Thread started
Thread terminated
Main program terminated
```

Conclusion

The program successfully created a thread using Python's threading module. The thread executed the `worker()` function concurrently within the same process and terminated after completing its task. The main program used `join()` to wait for the thread to finish before printing its final message.

How to Run on Linux (Ubuntu)

Step 1 — Open Terminal

Press `Ctrl+Alt+T` or search 'Terminal' in your applications.

Step 2 — Create process lab.py

```
Type: nano process_lab.py
```

Paste the process code, then press `Ctrl+O` to save and `Ctrl+X` to exit.

Step 3 — Run process lab.py

```
Type: python3 process_lab.py
```

Step 4 — Create thread lab.py

```
Type: nano thread_lab.py
```

Paste the thread code, then press `Ctrl+O` to save and `Ctrl+X` to exit.

Step 5 — Run thread lab.py

```
Type: python3 thread_lab.py
```

Note: Take a screenshot of your terminal output after running each program and attach it to your lab report submission.